

Licenciatura Engenharia Informática e Multimédia

Instituto Superior de Engenharia de Lisboa

Ano letivo 2023/2024

Inteligência Artificial para Sistemas Autónomos

Relatório: Entrega 1

Nome: Marta Ferreira Nº 50775 41D

Docente: Luís Morgado

Data: 28 de Abril de 2024

Índice

Conteúdo

Introdução 3
Enquadramento Teórico 4
Projeto: Parte 1 4
Subsistema Ambiente 5
Subsistema Agente 6
Subsistema Ambiente do Jogo 8
Subsistema Personagem 9
Subsistema Máquina de Estados 10
Projeto: Parte 2 11
Reação – Arquitetura 11
Esquemas Comportamentos Reativos 12
Controlo Reativo 13
Comportamento Explorar 14
Controlo Reativo - Reações 14
Comportamentos Compostos 15
EstimuloAlvo e EstimuloObst 15
Resposta a Estímulos 16
Conclusão 17

Introdução

O objetivo deste projeto é criar e implementar um jogo que irá utilizar as linguagens de programação Java e Python e irá demonstrar como é que a inteligência artificial é feita nos jogos.

Este jogo irá aplicar diversos tópicos falados em aula que demonstram o básico da implementação de inteligência artificial e os seus pontos fortes.

Ao longo deste relatório esses tópicos serão abordados de maneira mais complexa

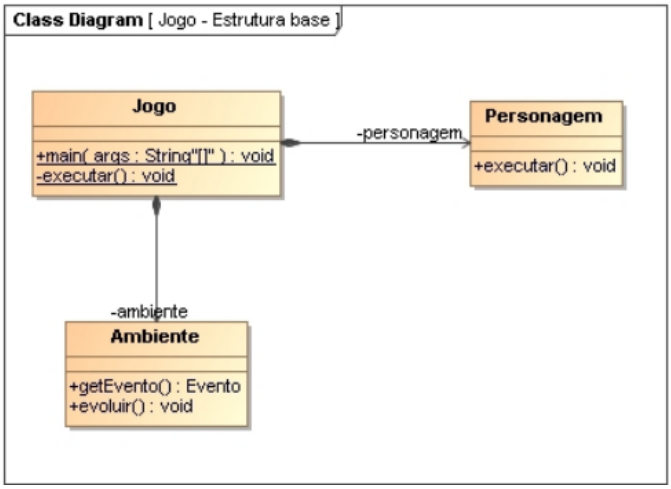
Enquadramento Teórico

O enquadramento teórico deste trabalho está dividido entre as diversas partes do mesmo, sendo assim mais fácil comparar a teoria com a aplicação do código nos diversos momentos ao longo do trabalho.

Projeto: Parte 1

Para a primeira parte deste projeto, pretende-se implementar um jogo na linguagem de programação Java que permita a interação entre um jogador humano e um personagem virtual, a mesma sendo realizada em modo de texto, utilizando a consola.

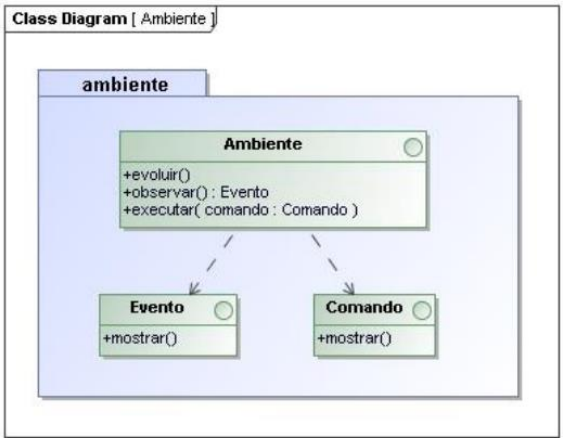
Com o objetivo de implementarmos o jogo, foram utilizados os diagramas de classes e diagramas de sequência fornecidas pelo docente. Começamos por implementar o seguinte modelo de estruturas (diagrama de classes).



Este diagrama é constituído por três classes fundamentais: classe Jogo, classe Personagem e classe Ambiente, cada uma irá ser aprofundada ao longo desta parte.

Na classe Jogo irá ser executado o nosso jogo com o ambiente e personagem criados no main da mesma. Este ambiente e personagem vão evoluir ao decorrer do jogo até que o mesmo acabe quando o evento do Ambiente for *TERMINAR*

Subsistema Ambiente



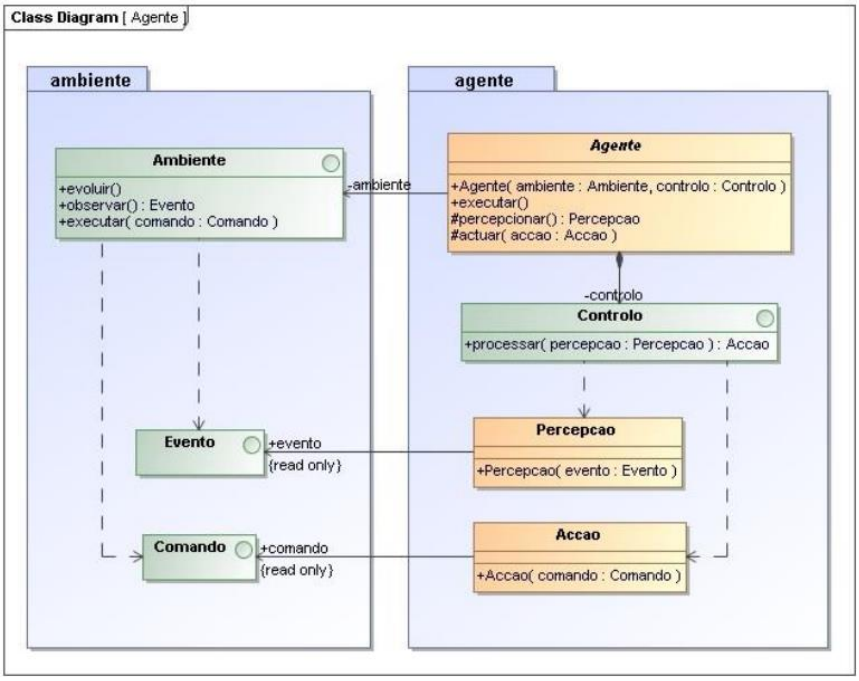
Começamos pela implementação do package ambiente e de todas as interfaces que fazem parte do mesmo, as interfaces são classes abstratas utilizadas para lidar com complexidades no código, nas mesmas ocorre a identificação de características comuns de diferentes módulos que irão utilizar as mesmas. A utilização de interfaces apresenta uma menor complexidade que permite realçar aquilo que é essencial, omitindo tudo o que não seja relevante, simplificar por redução a complexidade da descrição e o foco nos aspetos mais relevantes para o contexto da descrição.

O package ambiente é constituído por três interfaces: Ambiente, Evento e Comando.
A primeira é utilizada na criação de ambiente que ao decorrer do tempo irá evoluir para um novo estado devido ao seu método evoluir(), esta classe apresenta outros dois métodos observar() que tal como indica o nome irá observar e criar um novo Evento e por fim o método executar(Comando comando), que irá executar uma ação a partir de um determinado comando.

A interface Evento apresenta apenas um método, mostrar() em que a sua única função irá ser exibir as informações de um determinado evento que implemente a interface.

Por fim, a interface Comando tem um uso similar ao Evento, porem a mesma ao invés de exibir um evento com o seu método mostrar(), irá apresentar os comandos que a classe que o implementar terá.

Subsistema Agente



O package agente representa um agente inteligente, o mesmo tem a habilidade de percecionar, processar e atuar durante todos os momentos do jogo devido às características que o mesmo apresenta.

Um agente inteligente apresenta quatro características principais:

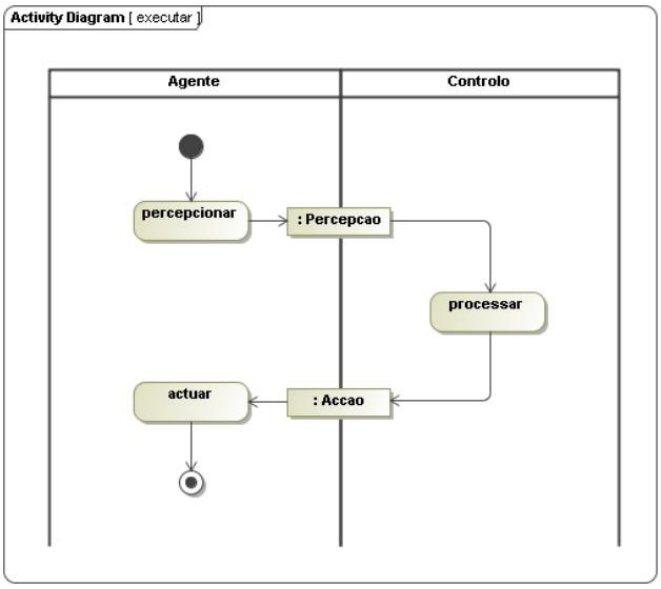
- Autonomia: Permite um sistema operar por si próprio de modo independente a outros sistemas.
- Reatividade: Permite um sistema reagir aos estímulos que o ambiente proporciona.
- Pró-Atividade: Permite que o sistema toma decisões de acordo com os seus objetivos
- Sociabilidade: Permite que um sistema interaja com outros agentes com a finalidade de concretizar objetivos individuais ou comuns entre ambos.

Todas estas características estão associadas à realização dos objetivos que o agente possui, expressos nas funções que realiza.

Existem três arquiteturas diferentes para um agente, a primeira é o modelo reativo que apresenta um paradigma comportamental, ou seja, o comportamento do sistema é gerado de forma reativa e baseia-se entre associações entre estímulos (que vêm da perceção) e respostas (que vêm das ações), a mesma possui objetivos implícitos visto que não apresenta mecanismos de deliberação. O modelo seguinte é o modelo deliberativo que apresenta um paradigma simbólico, ou seja, o comportamento do agente é gerado utilizando mecanismos de deliberação (raciocínio e tomada de decisão) tendo como finalidade objetivos explícitos. Por fim temos o modelo híbrido em que o comportamento do agente vai ser gerado com base no processamento que possuiu partes reativas e deliberadas.

Para podermos aplicar toda a teoria em código, foi criada a classe Agente que tem como objetivo representar um agente que irá poder interagir com o ambiente. Esta classe tem um construtor onde irá ser inicializado um ambiente e um controlo que irão ser passados como argumento.

Também possuiu o método executar que vai tal como o nome indica executar todos os métodos do agente, este método foi extraído a partir de um diagrama de atividades fornecido no enunciado do trabalho que se encontra em baixo.

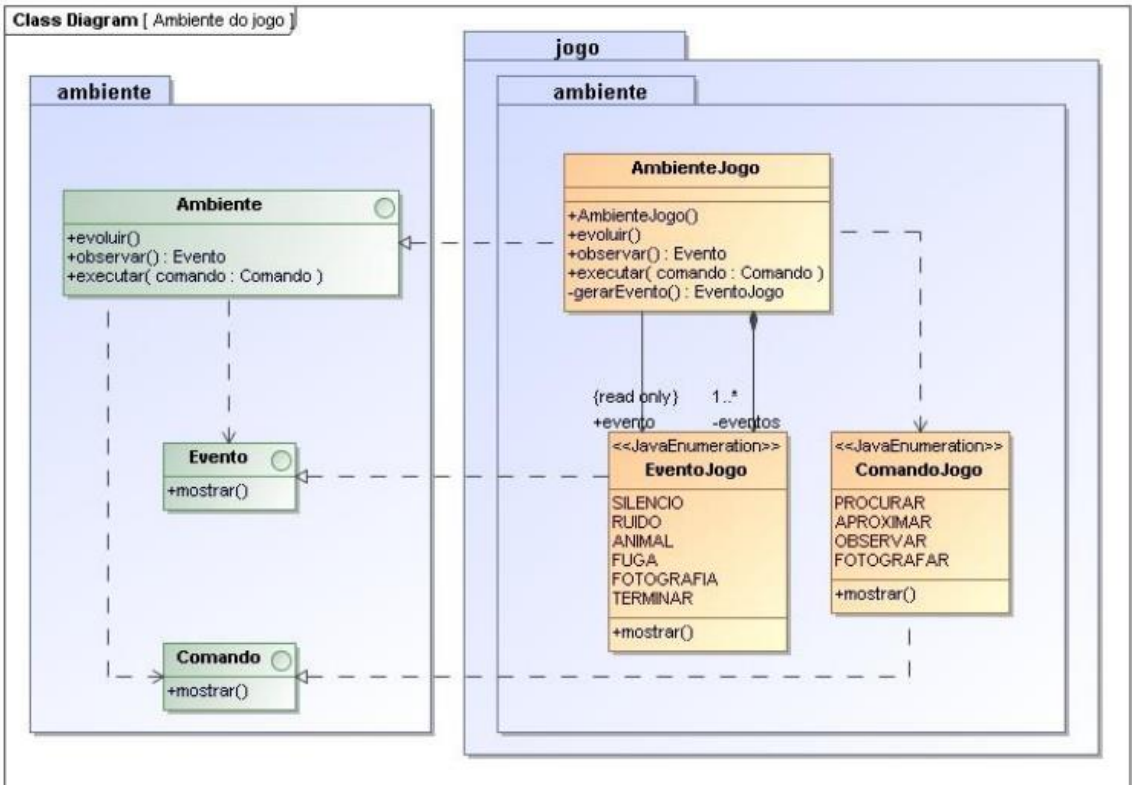


O inicio da atividade é representada por uma bola pintada de preto, as atividades são representadas por retângulos com pontas redondas, essas atividades serão os métodos percepcionar(), processar() e actuar(). Cada atividade é seguida por um Object node que são informações transferidas entre atividades, a primeira vai gerar um objeto do tipo Percepcao que irá ser utilizado pela atividade seguinte, a mesma irá criar um objeto do tipo accao que irá ser utilizado na atividade actuar, finalizando seguidamente a atividade representada pela bola branca com uma bola preta dentro da mesma.

No package agente está presente duas outras classes, Percepcao e Accao, ambas possuem um construtor que recebe um evento e um comando respetivamente e um getter de Evento (no caso da Percepcao) e um getter de Comando (no caso da Accao). A percepcao irá representar a percepção que o agente tem no ambiente e a accao representa as diversas ações que o agente terá, onde cada uma será associada a um comando diferente.

Por fim, este package possuiu uma interface Controlo que faz com que a classe agente processa uma percepcao e transforma-a numa accao utilizando o método processar().

Subsistema Ambiente do Jogo



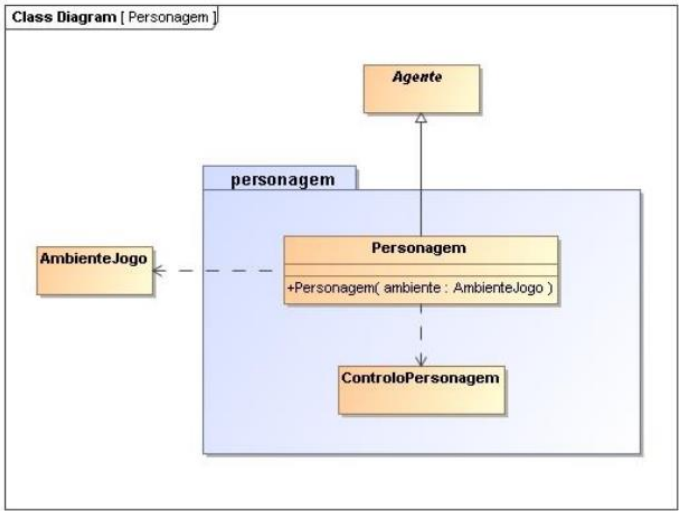
O subsistema Ambiente do Jogo é constituído por três classes, AmbienteJogo que implementa a interface anteriormente referida, Ambiente, EventoJogo que implementa a interface Evento e por fim ComandoJogo que implementa a interface Comando, este package irá gerar e controlar todos os aspetos importantes do ambiente do Jogo.

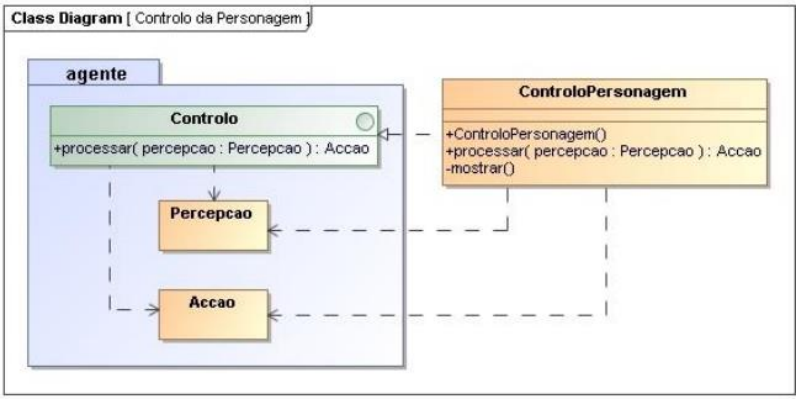
A classe EventoJogo é uma classe enum que implementa a interface Evento e que é utilizada para definir constantes que representam um valor fixo e diferentes entre si, cada evento enumerado será uma constante do tipo EventoJogo, as constantes criadas são: *SILENCIO, RUIDO, ANIMAL, FUGA, FOTOGRAFIA, TERMINAR*. Este método também tem a classe mostrar que vem da interface e que dá print na consola o evento que está a decorrer.

A classe ComandoJogo é idêntica à de cima, porém implementa a interface Comando e as constantes vão ser do tipo ComandoJogo e irão representar as seguintes ações: *PROCURAR, APROXIMAR, OBSERVAR, FOTOGRAFAR*. Tal como o EventoJogo esta classe também implementa um método mostrar da sua interface que irá dá print na consola da ação atual.

Por fim foi implementada a classe AmbienteJogo que implementa a interface Ambiente, a mesma possui um construtor onde é criado um HashMap<String, EventoJogo>() que associa uma tecla (s, r, a, f, o, t) a um dos Eventos do jogo definidos na classe EventoJogo. Esta classe também apresenta um método evoluir() que vai chamar o método gerarEvento() que por sua vez irá pedir o input do utilizador para o próximo evento e de acordo com a letra escolhida, o evento irá mudar para o evento que representa a letra escolhida. O AmbienteJogo também possui o método observar() que dá print do evento e retorna-o, o método executar() que recebe um comando e dá print do mesmo e por fim um getter getEvento() que retorna o evento atual.

Subsistema Personagem





O package Personagem tem como objetivo a criação de um personagem do tipo Agente, este personagem irá ser responsável pela criação e gestão dos comportamentos da personagem em relação ao jogo.

O mesmo apresenta uma classe Personagem que estende o Agente e é constituída apenas pelo construtor, o mesmo recebe um ambiente que irá ser usado na criação do Agente que também vai precisar de uma instância do tipo ControloPersonagem.

Este package também possuiu uma classe ControloPersonagem que implementa a interface Controlo, nesta classe irá ser implementa todos os controlos possíveis para o personagem. Os mesmos são feitos levando em consideração os Estados (procura, inspeção, observação e registo) e as Ações (procurar, aproximar, observar e fotografar) que são inicializados no construtor no mesmo. Irá ser utilizada uma Máquina de Estados maqEst que irá ser explicada com o decorrer do relatório. Também no construtor são criadas transições de acordo com a seguinte tabela fornecida pelo docente

Função de transição de estado

$\delta : Q \times \Sigma \rightarrow Q$

Estado	Evento	Novo estado
Procura	Animal	Observação
Procura	Ruido	Inspecção
Procura	Silencio	Procura
Inspecção	Animal	Observação
Inspecção	Ruido	Inspecção
Inspecção	Silencio	Procura
Observação	Fuga	Inspecção
Observação	Animal	Registo
Registo	Animal	Registo
Registo	Fuga	Procura
Registo	Fotografia	Procura

Tabela de transição de estado

Função de saída

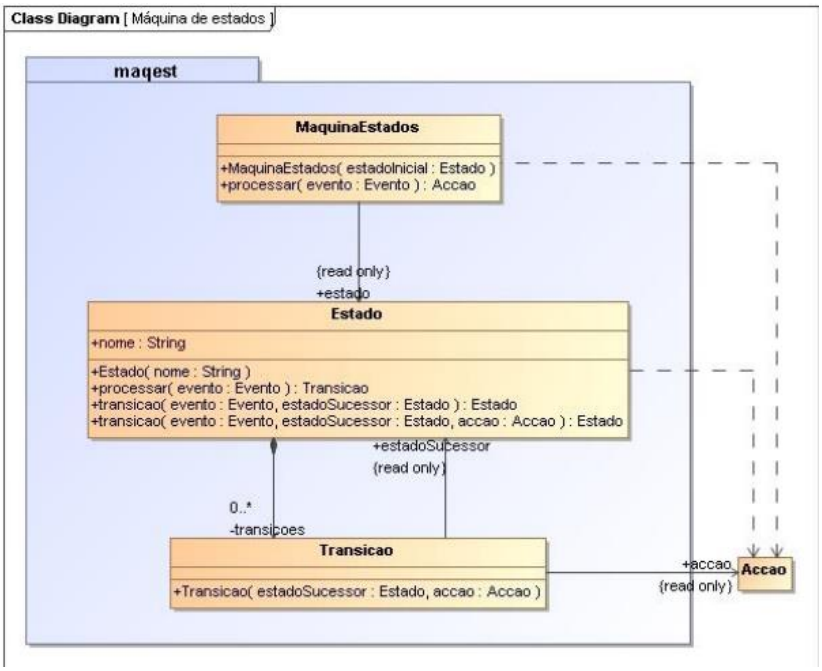
$\lambda : Q \times \Sigma \rightarrow Z$

Estado	Evento	Acção
Procura	Animal	Aproximar
Procura	Ruido	Aproximar
Procura	Silencio	Procurar
Inspecção	Animal	Aproximar
Inspecção	Ruido	Procurar
Inspecção	Silencio	
Observação	Fuga	
Observação	Animal	Observar
Registo	Animal	Fotografar
Registo	Fuga	
Registo	Fotografia	

Tabela de acção de estado

O ControloPersonagem também possui um método mostrar() que dá print do estado atual, um getter getEstado() que retorna o estado e o processar() que recebe uma perceção, e irá processá-la utilizando a máquina de estados transformando-a numa ação.

Subsistema Máquina de Estados



Uma máquina de estados é um mecanismo computacional descrito pelo modelo formal de computação, sua implementação física implica que exista um número finito de estados. A mesma é constituída por entradas representado pelo símbolo de somatório, saídas representadas por um Z, um circuito combinatório que possui a evolução de estado e a geração de saída representadas pelas letras gregas delta e lambda respetivamente e por uma memória de estado que tal como o nome indica guarda o estado seguinte e retorna o estado atual (representados por q' e q respetivamente).

Existem duas maneiras distintas de obter a função de saída:

- Máquinas de Mealy em que a função de saída depende das entradas
- Máquinas de Moore em que a função de saída não depende das entradas

Este package é constituído por três classes, MaquinaEstados, Estado e Transição. A MaquinaEstados contem um construtor, um getter getEstados() e um método processar(), o mesmo recebe um evento e processa-o retornando a transição necessária, de acordo com essa transição, o estado é atualizado para o estado seguinte e retorna a ação necessária para essa transição ocorrer.

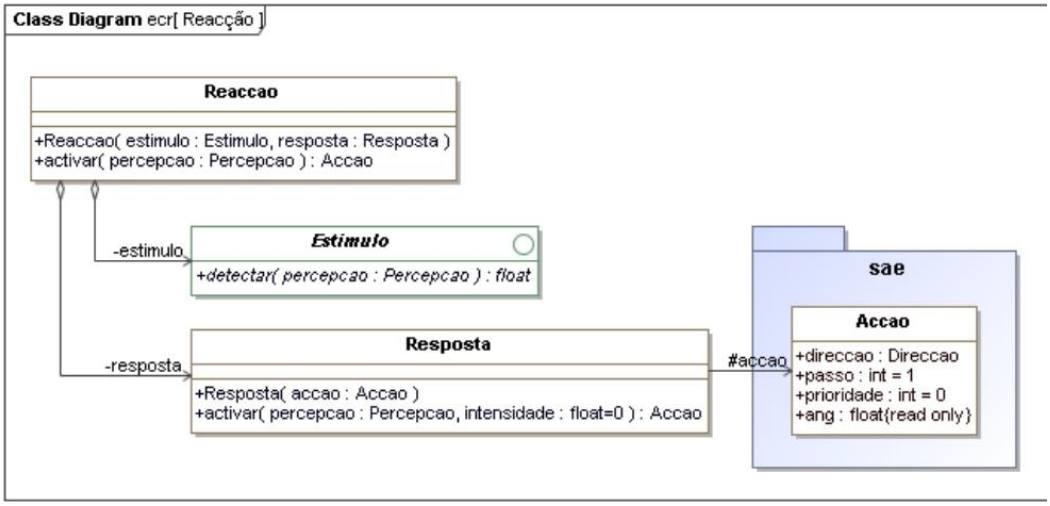
A classe Estado vai ser utilizada para criar todos os estados do jogo, o mesmo é uma representação da evolução de um sistema ou parte dele. Os estados vão poder evoluir dependendo de certos eventos do ambiente e/ou ações dos personagens.

A classe Transição irá ser utilizada para a criação de todas as transições que vão ser necessárias, para criarmos uma transição irá ser preciso o estado sucessor e uma ação.

Projeto: Parte 2

Esta parte do projeto tem como foco o desenvolvimento de agentes reativos em Python e com o auxílio da biblioteca SAE. Para este agente irá ser escolhida a arquitetura de modelo reativo que como falado anteriormente, é gerado de forma reativa com base em associações entre a perceção e a ação e terá objetivos implícitos.

Reação – Arquitetura

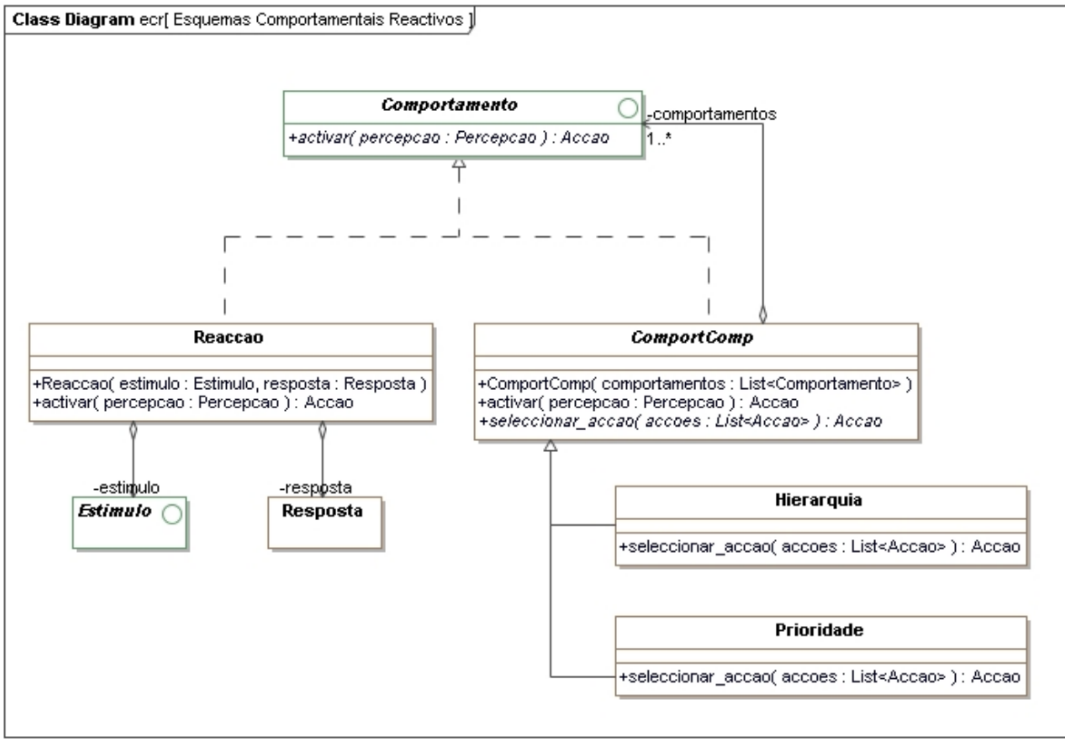


Primeiramente é criada a classe Reaccao que irá estender a classe Comportamento, que irá ser explicada mais para a frente, esta classe representa o módulo que associa estímulos a respostas, utilizando perceções que são agregados de estímulos. Na arquitetura de agentes reativos a reação utiliza uma regra de estimulo-resposta, ou seja, nesta arquitetura as ações são ativadas diretamente em função das perceções. Esta classe possuiu um método activar(percepcao) onde irá ser calculada a intensidade da reação a partir do detetor de estímulo da perceção, caso essa intensidade seja maior que 0 a mesma irá ativar a resposta e retornar uma ação.

Estímulo é uma interface que faz parte dos elementos dos Agentes Reativos, o mesmo define qual será a informação que será necessária para ativar uma reação. Esta classe contém um método detectar(percepcao), que irá ser implementado nas classes que usarem esta interface.

Por fim, Resposta é uma classe que irá definir uma reposta de acordo com determinados estímulos, em termos da ação e da respetiva prioridade que irá ser explicada brevemente. Esta classe possui um método activar(percepcao, intensidade = 0), caso no momento de ativação da resposta não seja dada uma intensidade, a mesma será definida como 0. Esta classe também utiliza objetos do tipo Accao, uma classe que vem da biblioteca dada pelo docente.

Esquemas Comportamentos Reativos



Um agente reativo é comporto por conjuntos de reações organizadas em vários módulos comportamentais e designados como comportamentos, levando à redução de complexidade e à facilitação do desenvolvimento e manutenção do agente. Tem-se que um comportamento é um conjunto de ações que por sua vez estão relacionadas entre si. O mesmo realiza o encapsulamento das reações internas levando ao acoplamento reduzido que ajuda a diminuir a complexidade do código. Um comportamento pode ser composto por outros comportamentos sendo assim chamado de Comportamento Composto.

Um comportamento composto agrega conjuntos de comportamentos e utiliza um mecanismo de seleção de ação para escolher qual ação irá ser realizada de acordo com os vários comportamentos. Essa seleção de ação pode ser de Hierarquia ou Prioridade.

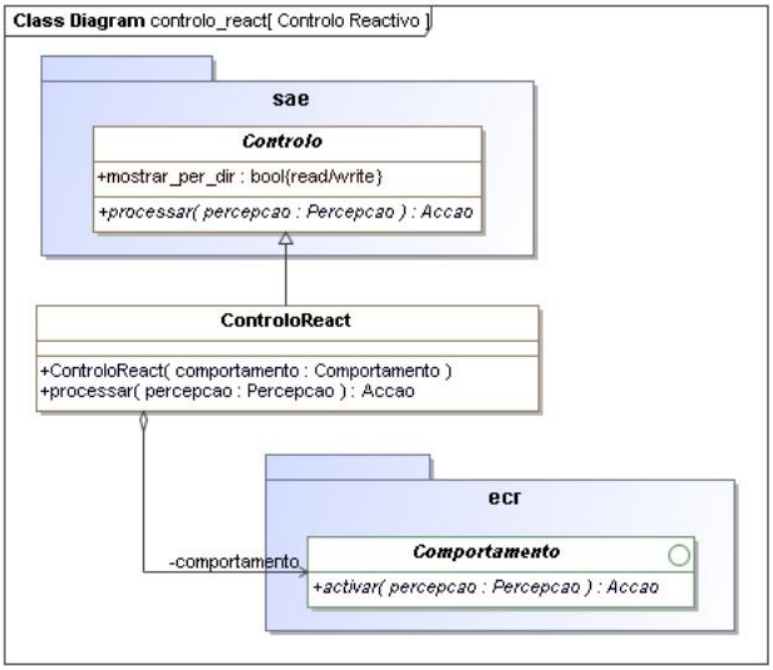
Na hierarquia (representada pela classe Hierarquia) os comportamentos estão de forma hierárquica em que os comportamentos estão ordenados por importância, a primeira reação irá retornar uma ação e sempre que possível irá suprimir e substituir as ações retornadas pelas outras reações.

Em Prioridade (representada pela classe Prioridade) as ações são selecionadas de acordo com uma prioridade que lhes é associada que poderá variar ao longo do tempo.

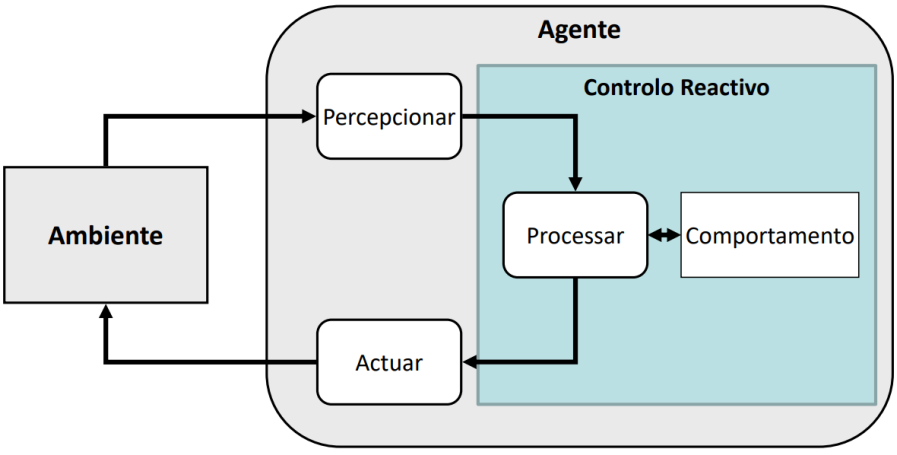
Irá ser criada uma interface Comportamento com o intuito de ser um comportamento simples já estabelecido em cima.

Por fim é criada a classe ComportComp que implementa a interface Comportamento, esta classe representa um Comportamento Composto que já foi explicado anteriormente. Esta classe apresenta um método activar(percepcao) que irá criar um array vazio que irá conter todas as ações, e percorrer um array com todos os comportamentos o mesmo irá criar uma ação para cada comportamento e se a mesma existir adicioná-la ao array das ações.

Controlo Reativo

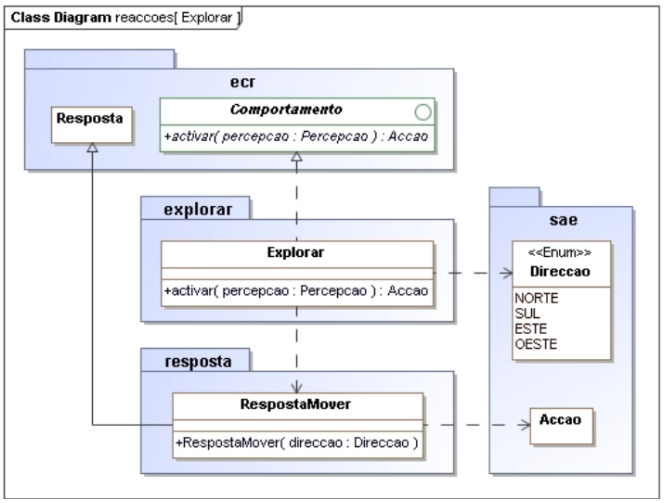


Para podermos realizar com sucesso uma arquitetura de agente, o processamento interno do mesmo pode ser feito com base num módulo de controlo. No nosso caso em que temos um agente reativo, esse controlo irá ser um controlo reativo em que o processamento das perceções vai ser realizado com base num módulo comportamental tal como indica a figura abaixo.



Vai ser criada a classe ControloReact que irá representar o controlo reativo do agente. A mesma possui um método processar() que concretiza a função de processamento interno que relaciona perceções com ações, ou seja vai processar uma perceção de acordo com o comportamento e vai retornar uma ação de acordo com o mesmo.

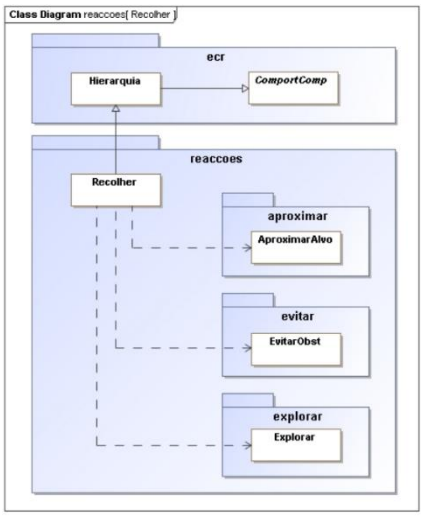
Comportamento Explorar



Este comportamento vai ser responsável pelo movimento aleatório do agente reativo. A classe Explorar implementa a interface Comportamento e vai criar uma RespostaMover com uma direção aleatória utilizando as direções fornecidas (*NORTE, SUL, ESTE, OESTE*) pela biblioteca SAE.

A classe RespostaMover estende a classe Resposta e no seu construtor vai criar uma ação com a direção aleatória para ser utilizada no construtor da classe pai.

Controlo Reativo - Reações



O nosso agente vai ter como objetivo principal recolher alvos (classe Recolher), para conseguir realizar essa missão o mesmo precisa de três reações diferentes: AproximarAlvo(), EvitarObst(), Explorar.

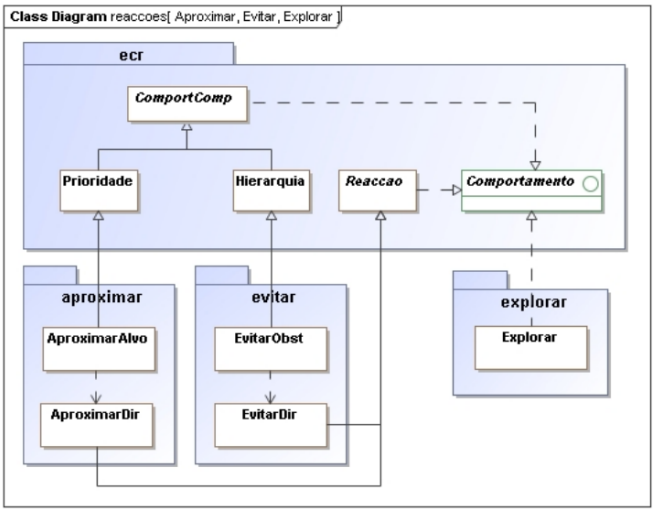
A classe Recolher vai então ser um comportamento composto que agrega o conjunto desses sub-comportamentos. O AproximarAlvo() aproxima-se de um alvo caso o mesmo exista, o EvitarObst() evita um obstáculo que esteja no seu caminho e o Explorar() vai ser utilizado quando não se é detetado nenhum alvo ou obstáculo para que o agente ande livremente. A classe Recolher vai ter uma seleção de ação hierárquica ou seja o agente vai sempre tentar realizar o primeiro comportamento e caso não consiga passa para o próxi

mo.

AproximarAlvo() é uma classe que representa um comportamento em que o agente irá aproximar-se do alvo, este é um comportamento de selecção de acção prioritária ou seja, o agente se tiver um alvo a norte perto e outro a este longe ele irá mover-se na direção do que estiver mais perto (neste caso a norte), esta prioridade faz com que o agente se mova num percurso mais otimizado indo sempre para o alvo mais perto do mesmo.

EvitarObst() é uma classe que representa um comportamento do tipo hierarquia que irá evitar obstáculos nas quatro direções que o mesmo se pode mexer. Esta classe irá criar uma lista de comportamentos agrupados de forma hierárquica para que o agente possa agir de forma adequada a cada obstáculo.

Comportamentos Compostos

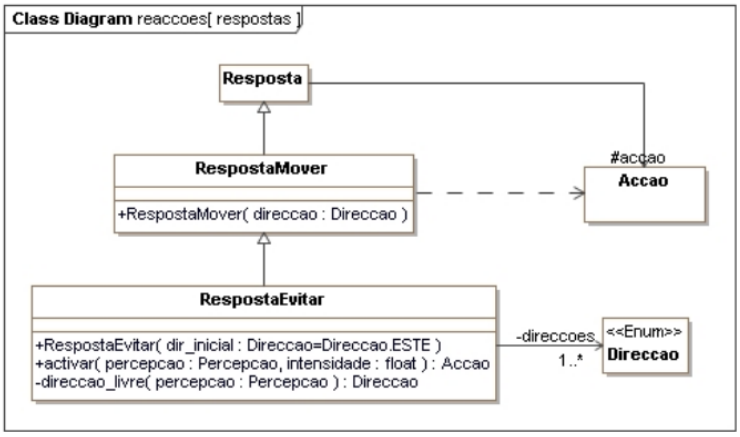


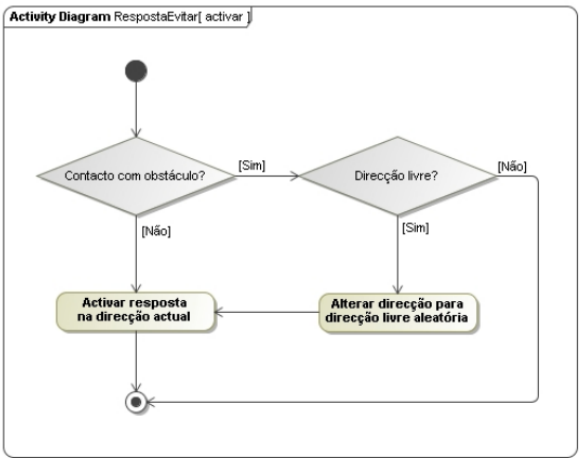
Os comportamentos AproximarAlvo() e EvitarObst() podem ser dividido em vários comportamentos AproximarDir(direção) e EvitarDir(direção) em que direção é uma das quatro direções possíveis para o movimento (NORTE, SUL, ESTE, OESTE), tornando-os comportamentos compostos. Estes novos comportamentos simples são escolhidos com uma seleção de ação de prioridade fazendo com que o percurso do agente até um alvo seja o mais rápido e eficaz possível, o mesmo irá acontecer com o EvitarDir() mas de forma a que o agente se afaste do obstáculo.

EstimuloAlvo e EstimuloObst

Estes estímulos irão ser utilizados pelas classes AproximarDir() e EvitarDir() para gerar uma ação para o agente e vai fazer com que o mesmo se mova numa determinada direção de acordo com alvos/obstáculos ao seu redor. Para isso será ser usado o método de detetar que no caso do EstimuloAlvo() vai detetar um alvo e retornar uma intensidade de acordo com a distância do agente ao mesmo, no EstimuloObst() vai detetar um obstáculo e retornar uma intensidade de 1.

Resposta a Estímulos





Para termos todas as respostas ao estímulos só falta implementar a classe RespostaEvitar(). Esta classe vai enviar respostas ao agente de modo que o mesmo evite objetos que estejam no seu caminho nunca batendo nos mesmos. O seu construtor é inicializado com a direção inicial a ESTE e uma lista que contem as quatro direções possíveis.

O método activar() desta classe vai verificar se o agente está em contacto com um obstáculo, se a verificação for positiva, irá ser chamado o método DireccaoLivre que vai fazer uma lista com todas as direções em que o agente se pode mover sem bater contra um obstáculo e vai retornar uma das mesmas escolhida aleatoriamente. Por fim essa direção aleatória passa a ser a direção em que o agente se vai mover, evitando assim ir contra um obstáculo.

Conclusão

Com a realização das duas primeiras partes deste projeto pudemos adquirir mais conhecimento sobre a implementação de sistemas inteligentes. Com a realização e documentação do código realizado em Java pudemos aprender mais sobre a utilização de Máquinas de estado na criação de ações inteligentes de um personagem. Com a parte de Python obtivemos um conhecimento mais extenso sobre os agentes reativos e o seu uso na criação de jogos com inteligência artificial.